

University of Algiers 1 - Faculty of Sciences

Department of Computer Science
Module: Semi-Structured Data (SSD)

Final Examination 2025/2026

Date 09/05/2026

Duration: 1h30

Student Name: _____ ID: _____ Group: _____ Grade: _____/20

Part I (3 pts)

We consider the XML file `recette_cuisine_algerie.xml` describing Algerian traditional recipes.

Study the following XML fragment and identify **six semantic or structural errors**.

```
<recipes>
  <recipe id="R01" type="dish" time="-30">
    <name>Couscous Royal</name>
    <ingredient ref="I9"/>
    <ingredient ref="I1"/>
    <price>1200</price>
    <note>12</note>
  </recipe>
  <recipe id="R01" type="dessert">
    <name></name>
    <ingredient ref="I2"/>
    <price>-150</price>
  </recipe>
  <ingredient id="I1">Semolina</ingredient>
  <ingredient id="I2">Honey</ingredient>
  <chef id="C1"/>
</recipes>
```

No	Error	Correction
1	Duplicate id="R01"	Use unique ids for each recipe
2	Negative time value "-30"	Use a positive cooking time
3	Ingredient ref="I9" does not exist	Reference an existing ingredient id
4	Empty name element	Add a valid recipe name
5	Negative price "-150"	Price must be positive
6	Empty chef element	Add chef information/content

Part II (4 pts)

Write the DTD corresponding to `recette_cuisine_algerie.xml`.

The document contains recipes, ingredients and chefs. Each recipe contains a name, one or more ingredient references, a price, a note and optional preparation steps.

```

<!ELEMENT recipes (recipe+, ingredient+, chef+)>
<!ELEMENT recipe (name, ingredient+, price, note, step*)>
<!ELEMENT name (#PCDATA)>
<!ELEMENT ingredient (#PCDATA)>
<!ELEMENT price (#PCDATA)>
<!ELEMENT note (#PCDATA)>
<!ELEMENT step (#PCDATA)>
<!ELEMENT chef EMPTY>
<!ATTLIST recipe id ID #REQUIRED type CDATA #REQUIRED time CDATA #IMPLIED>
<!ATTLIST ingredient id ID #IMPLIED ref IDREF #IMPLIED>
<!ATTLIST chef id ID #REQUIRED>

```

Part III (4 pts)

- a) Explain the difference between `xsd:sequence`, `xsd:choice`, and `xsd:all` in XML Schema (XSD).
`xsd:sequence` imposes an order between elements, `xsd:choice` allows only one element among many, `xsd:all` allows elements in any order.
- b) Explain the concept of defining elements outside their parent structure and referencing them using `ref` in XML Schema (XSD).
Elements can be declared globally and reused inside other elements using `ref` attribute.
- c) Write the XSD declaration of a **simple restricted element** validating the note: decimal between 0 and 10,

```

<xs:element name="note">
  <xs:simpleType>
    <xs:restriction base="xs:decimal">
      <xs:minInclusive value="0"/>
      <xs:maxInclusive value="10"/>
    </xs:restriction>
  </xs:simpleType>
</xs:element>

```

- d) Write the XSD declaration of the **element** `receipe` containing: name, one or more ingredient, prix, note, optional etape.

```

<xs:element name="recipe">
  <xs:complexType>
    <xs:sequence>
      <xs:element name="name" type="xs:string"/>
      <xs:element name="ingredient" type="xs:string"
        minOccurs="1" maxOccurs="unbounded"/>
      <xs:element name="price" type="xs:decimal"/>
      <xs:element name="note" type="xs:decimal"/>
      <xs:element name="step" type="xs:string" minOccurs="0"/>
    </xs:sequence>
  </xs:complexType>
</xs:element>

```

Part IV (3 pts)

Write a complete XQuery FLWOR expression that generates a report: root element <rapport>, for each recipe: name, number of ingredients, price, if note > 8 add <excellent> node.

```

<rapport>{
for $r in //recipe
let $name := $r/name
let $nb := count($r/ingredient)
let $price := $r/price
let $note := $r/note

return
  (
    <recette>
      <name>{$name}</name>
      <ingredients>{$nb}</ingredients>
      <price>{$price}</price>
    </recette>,

    if ($note > 8) then
      <excellent>
        <name>{$name}</name>
        <note>{$note}</note>
      </excellent>
    else ()
  )
}

```

```
}</rapport>
```

Part V (4 pts)

Write the XPath expressions answering the following requests.

1. Select the names of recipes having price greater than 1000.
`//recipe[price > 1000]/name`
2. Select all recipes containing more than one ingredient.
`//recipe[count(ingredient) > 1]`
3. Return the id of the recipe having the maximum note.
`//recipe[note=max(//recipe/note)]/@id`
4. Counts all recipes having a price greater than 1000 OR a note greater than 9.
`count(//recipe[price > 1000 or note > 9])`
5. Select all elements having at least one attribute.
`//*[@*]`

Part VI (2pts)

Question: Write a XSLT code to display recipes sorted by descending note with columns: name, type, price, note.

Answer:

```
<xsl:for-each select="recipe">
  <xsl:sort select="note" order="descending"/>
  <tr>
    <td><xsl:value-of select="name"/></td>
    <td><xsl:value-of select="@type"/></td>
    <td><xsl:value-of select="price"/></td>
    <td><xsl:value-of select="note"/></td>
  </tr>
</xsl:for-each>
```